instead, order the edges to be added or removed at each step in such a way to guarantee a linear number of steps (therefore using the first of the strategies above).

Anyway, by avoiding to compute things twice, we guarantee that we will examine each distinct subgraph at most once, so that the number of steps becomes bounded by the number of possible subgraphs: for a graph with `n` vertices and `m` edges, there are $2^n$ *induces subgraphs* (because there are $2^n$ subset of vertices) and $2^m$ spanning subgraphs (considering subset of edges); the total number of possible subgraphs is thus bounded by $2^{n+m}$, which is better than factorial, but still too large to consider the algorithm usable for graphs with more than ~20 vertices.

## Listing 15.7 Planarity Testing with cache

```
function isPlanar(graph, cache={})
  if graph in cache then return (cache[graph],
cache)                #1
  if |graph.vertices| < 5 then return (true, cache)
  if violatesEulerConstraints(graph) then
                             #2
    cache[graph] ← false
    return (false, cache)
  if isComplete(graph) or isNonPlanarCompleteBipartite(graph)
then   #3
    cache[graph] ← false
    return (false, cache)
  for v in graph.vertices do
                                    #4
    subG ←
graph.remove(v)
```